



LI.FI

LI.FI Security Review

SupersetFacet.sol(v1.0.0)

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

July 1, 2026

Contents

1	About Researcher	2
2	Disclaimer	2
3	Scope	2
4	Risk classification	2
4.1	Impact	2
4.2	Likelihood	3
4.3	Action required for severity levels	3
5	Executive Summary	3
6	Findings	4
6.1	Low Risk	4
6.1.1	amountOutMin == 0 is accepted	4
6.2	Informational	4
6.2.1	Positive-slippage scaling uses an unchecked-magnitude multiply	4
6.2.2	Linear amountOutMin scaling ignores AMM price impact	4
6.2.3	Chain/EID admin config accepts zero entries	4
6.2.4	Value-bearing refund/fallback addresses are unbound and hidden from clear signing	5
6.2.5	Superset path token is not bound to bridgeData.sendingAssetId	5

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast and Angles

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Facets/SupersetFacet.sol(v1.0.0)
- src/Errors/GenericErrors.sol(v1.0.4)
- src/Interfaces/ISupersetHubPoolManager.sol(v1.0.0)
- src/Interfaces/ISupersetSpokePoolManager.sol(v1.0.0)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

- High** leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium** global losses <10% or losses to only a subset of users, but still unacceptable.
- Low** losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

- High** almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium** only conditionally possible or incentivized, but still relatively likely
- Low** requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

- Critical** Must fix as soon as possible (if already deployed)
- High** Must fix (before deployment if not already deployed)
- Medium** Should fix
- Low** Could fix

5 Executive Summary

Over the course of 8 hours in total, [LI.FI](#) engaged with the [researcher](#) to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 6 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	6baa3f12
Audit Timeline	July 01, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Gas Optimizations	0
Informational	5
Total Issues	6

6 Findings

6.1 Low Risk

6.1.1 `amountOutMin == 0` is accepted

Context: [SupersetFacet.sol#L322](#)

Description: `_validateSupersetData()` never rejects a zero output floor, and the value is forwarded as-is (`SupersetFacet.sol:322`) into Superset's AMM, which uses it directly as `amountOutMinimum`. A bad or malicious quote can thus strip all destination-side slippage protection.

Recommendation: Reject `amountOutMin == 0` unless there is an explicit, documented dust-transfer mode.

LI.FI: Fixed in [bfd3488](#)

Researcher: Verified fix.

6.2 Informational

6.2.1 Positive-slippage scaling uses an unchecked-magnitude multiply

Context: [SupersetFacet.sol#L249](#)

Description: `SupersetFacet` multiplies two token amounts before dividing; for extreme (non-stablecoin-realistic) values the intermediate product would overflow and revert.

Not reachable for Superset's whitelisted stablecoins, and a revert only fails the caller's own transaction.

Recommendation: Optionally switch to `solady mulDiv` (already available) for defense-in-depth

LI.FI: Fixed in [02191ea](#)

Researcher: Verified fix.

6.2.2 Linear `amountOutMin` scaling ignores AMM price impact

Context: [SupersetFacet.sol#L249](#)

Description: After a source swap, the destination slippage floor is scaled linearly by `actualPostSwap / preSwapFloor`, but Superset enforces it as an AMM `amountOutMinimum` on concave virtual pools.

Because output is sub-linear in input, the scaled floor is over-stated, so the NatSpec's "percentage tolerance is preserved" invariant doesn't hold, the effective tolerance tightens as positive slippage grows. This is conservative (never under-protects funds) but can revert legitimate positive-slippage bridges.

Recommendation: Either document that the scaled floor is an intentionally-conservative approximation (errs strict), or if precise protection is required refund the positive swap slippage.

LI.FI: Documented in [2a6f8f3](#)

Researcher: Verified fix.

6.2.3 Chain/EID admin config accepts zero entries

Context: [SupersetFacet.sol#L133](#), [SupersetFacet.sol#L156](#)

Description: `initSuperset()` and `setChainIdToEid()` write each `{chainId, lzEid}` without rejecting zeros. Since `lzEid == 0` also means "unset" (and `getChainIdToEid()` reverts `UnsupportedChainId` on it), a zero EID emits a successful `ChainIdToEidSet` yet leaves the chain unusable; a zero `chainId` is a meaningless entry.

Recommendation: Reject `chainId == 0` and `lzEid == 0` in both functions.

LI.FI: Fixed in [2c0e442](#)

Researcher: Verified fix.

6.2.4 Value-bearing refund/fallback addresses are unbound and hidden from clear signing

Context: [clearSigningProposal.json#L2245](#)

Description: `refundAddress` is only checked nonzero (`SupersetFacet.sol:270`), yet on spoke-origin async failure Superset refunds the full input to it.

A malicious quote can set an attacker refund address and force failure via an impossible `amountOutMin` or near-expired `deadline` and the user can't catch it, since clear signing shows only amount/destination/recipient while omitting `refundAddress`, `fallbackEoA`, `amountOutMin`, `deadline`, `path`, and `lzFee` (`config/clearSigningProposal.json:2245,5585`).

Recommendation: Bind the refund/fallback fields to an authenticated source user or trusted signed quote, and expose them in the clear-signing entries.

LI.FI: Acknowledged. The clear-signing omission isn't Superset-specific: our bridge-agnostic generator surfaces only common `BridgeData/SwapData` fields today.

Researcher: Acknowledged.

6.2.5 Superset path token is not bound to `bridgeData.sendingAssetId`

Context: [SupersetFacet.sol#L266-L268](#)

Description: `SupersetFacet` deposits and approves `_bridgeData.sendingAssetId`, but Superset pulls the token resolved from `path.getFirstOmniToken()`, and the facet only checks `path.length != 0`. With a stale max allowance, an attacker deposits worthless token while pointing the path at stuck tokens in the diamond, and could steal it.

Since there are multiple ways across different facets to siphon locked funds out of the diamond, this issue is considered to have very low impact as diamond shouldn't hold any funds in the first place.

Recommendation: Resolve `path.getFirstOmniToken()` and require it equals `_bridgeData.sendingAssetId`; on the swap path also require `_swapData[last].receivingAssetId == _bridgeData.sendingAssetId`

LI.FI: Fixed in [070e2bb](#)

Researcher: Verified fix.